

CAP. 4 OTHER SUPERVISED LEARNING METHODS

References:

→ from textbook: cap. 8 - Tree-based methods; cap. 2 – Statistical learning

Prof. Luiz Alexandre Peternelli

Departamento de Estatística
Universidade Federal de Viçosa

Text and examples are taken from:

G. James et al., *An Introduction to Statistical Learning: with Applications in R*,
Springer Texts in Statistics, DOI 10.1007/978-1-4614-7138-7 5,
© Springer Science+Business Media New York 2013

2

Chapter 4 - content

- 4.1. Concepts about decision trees and the Random Forests)
- 4.2. K-nearest neighbor

Obs.: topics are numbered as they appear in the reference book.

Introduction

- These involve *stratifying* or *segmenting* the predictor space into a number of simple regions
- The set of splitting rules → summarized in a *tree* → called *decision tree methods*.
- Tree-based methods are simple and useful for interpretation. *However*, they typically are not competitive with other supervised learning approaches in terms of prediction accuracy
- How to increase the prediction accuracy? → *bagging*, *random forests*, and *boosting*
- Produce multiple trees → combined to yield a single consensus prediction → improvements in prediction accuracy, at the expense of some loss in interpretation.

8.1 The Basics of Decision Trees

- Applied to both *regression* and *classification* problems
- Remember the difference between both types of problems...

8.1.1 Regression Trees

- **Example:** Predicting Baseball Players' Salaries Using Regression Trees
- Data structure and data preparation:
 - Hitters data set to predict a baseball player's Salary based on
 - Years (the number of years that he has played in the major leagues) and
 - Hits (the number of hits that he made in the previous year).
 - first remove observations that are missing Salary values
 - log-transform Salary $\rightarrow$ typical bell-shape
 - OBS.: Salary is measured in thousands of dollars.

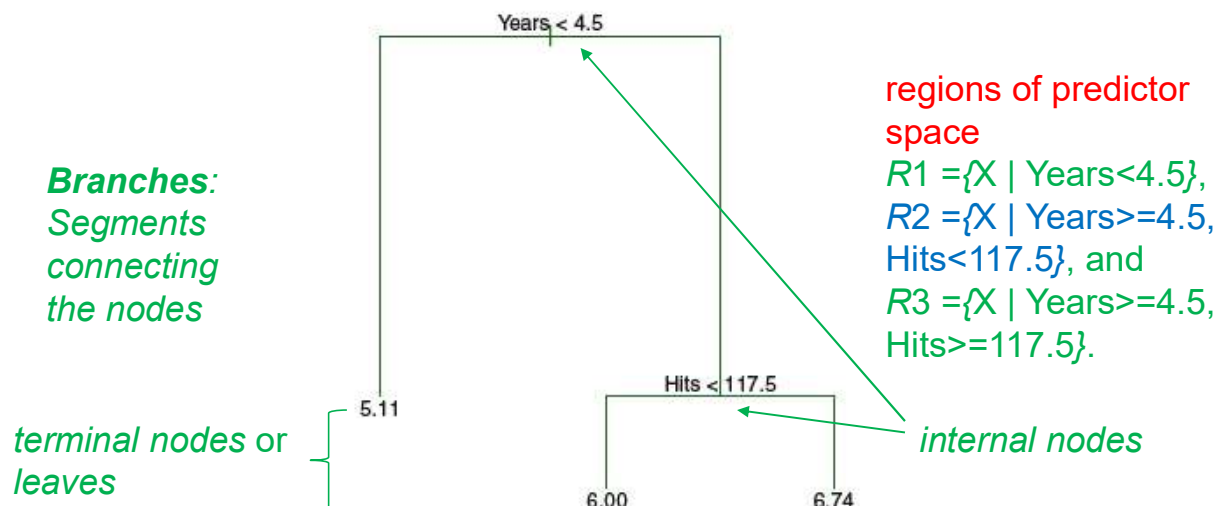


FIGURE 8.1. For the **Hitters** data, a regression tree for predicting the log salary of a baseball player, based on the number of years that he has played in the major leagues and the number of hits that he made in the previous year. At a given internal node, the label (of the form $X_j < t_k$) indicates the left-hand branch emanating from that split, and the right-hand branch corresponds to $X_j \geq t_k$. For instance, the split at the top of the tree results in two large branches. The left-hand branch corresponds to **Years** < 4.5, and the right-hand branch corresponds to **Years** $\geq$ 4.5. The tree has two internal nodes and three terminal nodes, or leaves. The number in each leaf is the mean of the response for the observations that fall there.

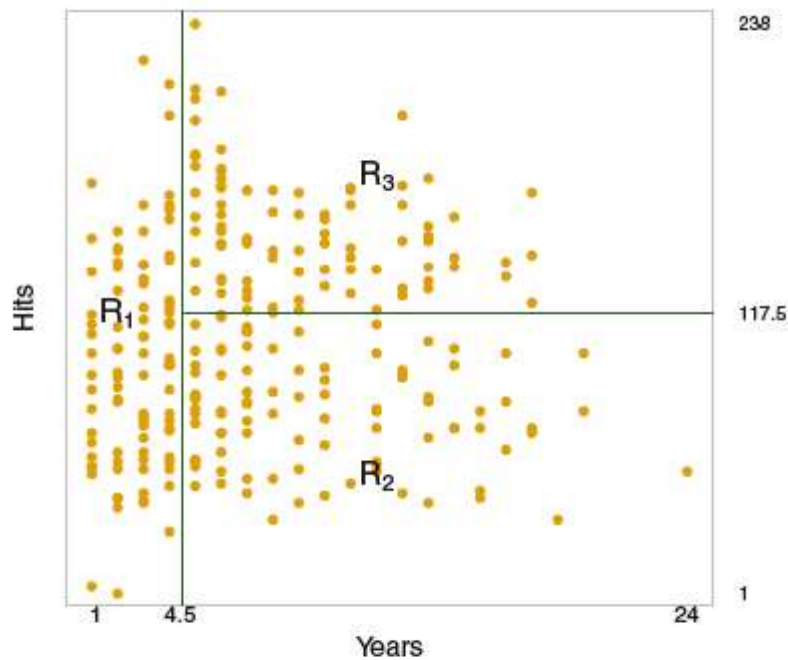


FIGURE 8.2. The three-region partition for the **Hitters** data set from the regression tree illustrated in Figure 8.1.

Prediction via Stratification of the Feature Space

- process of building a **regression tree**:
 - 1. We divide the predictor space—that is, the set of possible values for $X_1, X_2, \dots, X_p$ —into J distinct and non-overlapping regions, $R_1, R_2, \dots, R_J$.
 - the regions could have any shape. Boxes are easier to interpret.
 - 2. For every observation that falls into the region R_j , we make the same prediction, which is simply the **mean** of the response values for the training observations in R_j .
 - The goal: to find shapes $R_1, \dots, R_J$ that minimize the RSS

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

$\hat{y}_{R_j}$: mean response for the training observations within the j th box

Computational efforts:

- it is computationally infeasible to consider every possible partition of the feature space into J boxes.
- Solution: take a *top-down, greedy* approach that is known as *recursive binary splitting*.
- *top-down*: it begins at the top of the tree (a single region) → successively splits the predictor space; each split → two new branches further down on the tree.
- *Greedy*: at each step of the tree-building process, the *best* split is made at that particular step, rather than looking ahead and picking a split that will lead to a better tree in some future step.

Tree Pruning

- The process described before → good predictions on the training set, but is likely to overfit the data, leading to *poor test set performance*
- Because the resulting tree might be too complex.
- A smaller tree with *fewer splits* might lead to *lower variance* and *better interpretation* at the cost of a *little bias*
- *How*: grow a very large tree T_0 , → *prune it back* in order to obtain a *subtree*.
 - How to determine the best way to prune the tree?

- **Intuitively:** goal is to select a subtree that leads to the **lowest test error rate**
- Given a subtree $\rightarrow$ estimate its **test error** using cross-validation (CV) or the validation set approach
- However, estimating the CV error for every possible subtree would be too cumbersome.
- consider a sequence of trees indexed by a nonnegative **tuning parameter** α . For each value of α there corresponds a subtree $T \subset T_0$ such that

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

is as small as possible

The number of terminal nodes

- The α controls a trade-off between the subtree's complexity and its fit to the **training data**
- When $\alpha = 0 \rightarrow$ the subtree T will simply equal T_0
- as α increases, there is a price to pay for having a tree with many terminal nodes, and so the equation will tend to be minimized for a smaller subtree
- **select** α using a **validation set** or using **cross-validation**.
- **return to the full data set** and obtain the subtree corresponding to α .

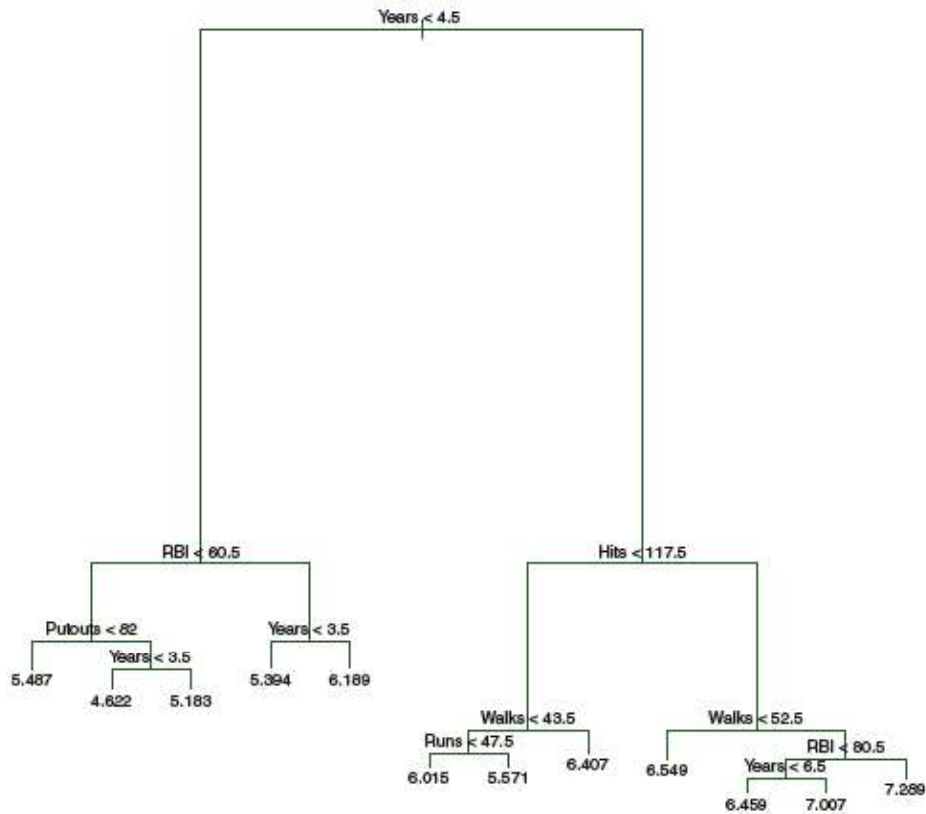


FIGURE 8.4. Regression tree analysis for the **Hitters** data. The unpruned tree that results from top-down greedy splitting on the training data is shown.

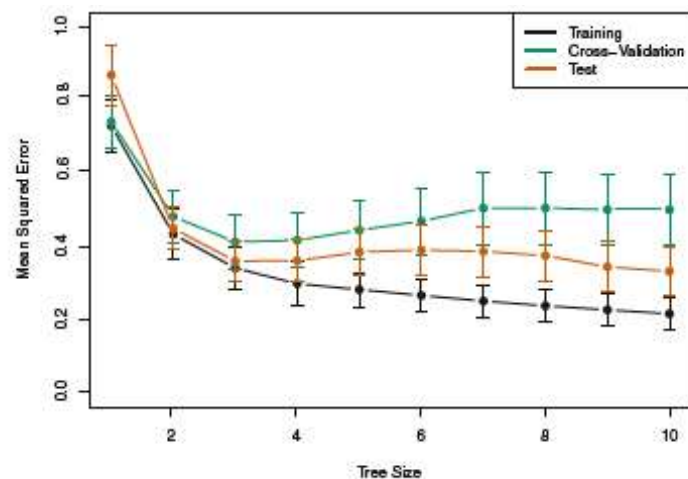


FIGURE 8.5. Regression tree analysis for the **Hitters** data. The training, cross-validation, and test MSE are shown as a function of the number of terminal nodes in the pruned tree. Standard error bands are displayed. The minimum cross-validation error occurs at a tree size of three.

8.1.2 Classification Trees

- very similar to a regression tree, but it is used to predict a **qualitative response** (the *most commonly occurring class* of the training observations) rather than a quantitative one (the mean response of the training observations)
- The task of growing a classification tree is quite similar to the task of growing a regression tree, but uses the *classification error rate instead of the RSS*.
- the classification error rate is simply the fraction of the training observations in that region that do not belong to the most common class:

$$E = 1 - \max_k (\hat{p}_{mk})$$

the proportion of training observations in the m th region that are from the k th class

- However, it turns out that classification error is not sufficiently sensitive for tree-growing, and in practice two other measures are preferable:

- **Gini index**

- total variance across the K classes
- a measure of node *purity*:
- small value $\rightarrow$ a node contains predominantly obs from a single class

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

- **Entropy**

- small value if the m th node is pure.

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}$$

- In fact, it turns out that the Gini index and the entropy are quite similar numerically

- When building a classification tree
 - either the Gini index or the entropy are typically used to evaluate the quality of a particular split, since these two approaches are more sensitive to node purity than is the classification error rate
- Any of these three approaches might be used when *pruning* the tree, but the classification error rate is preferable if prediction accuracy of the final pruned tree is the goal.
- See [Figure 8.6](#) and interesting discussion in the textbook

8.1.3 Trees Versus Linear Models

- linear regression assumes a model of the form

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j$$

- regression trees assume a model of the form

$$f(X) = \sum_{m=1}^M c_m \cdot 1_{(X \in R_m)}$$

- Which model is better? It depends on the problem at hand. See example...

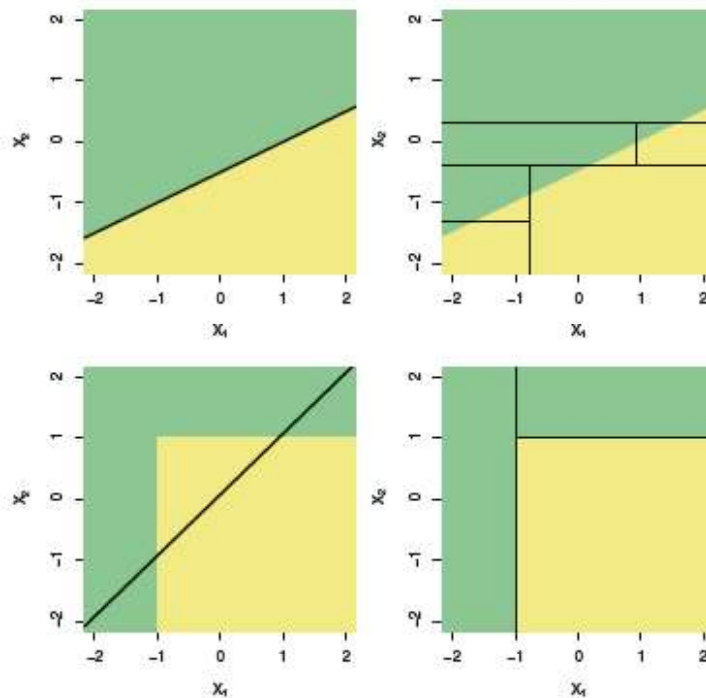


FIGURE 8.7. Top Row: A two-dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assumes a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right). Bottom Row: Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right).

8.1.4 Advantages and Disadvantages of Trees

- Trees are very easy to explain to people. In fact, they are even easier to explain than linear regression!
- Some people believe that decision trees more closely mirror human decision-making than do the regression and classification approaches.
- Trees can be displayed graphically, and are easily interpreted even by a non-expert.
- Trees can easily handle qualitative predictors without the need to create dummy variables.
- trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches.
- trees can be very non-robust. In other words, a small change in the data can cause a large change in the final estimated tree.

8.1.4 Advantages and Disadvantages of Trees

- Trees are very easy to explain to people. In fact, they are even easier than linear models.
- Some of the most powerful machine learning algorithms, such as *bagging*, *random forests*, and *boosting*, are based on trees. By aggregating many decision trees, using methods like these, the predictive performance of trees can be substantially improved.
- Trees can be very non-robust. In other words, a small change in the data can cause a large change in the final estimated tree.

8.2 Bagging, Random Forests, Boosting

8.2.1 Bagging

- construct B regression trees using B bootstrapped training sets, and average the resulting predictions.
- These trees are grown deep, and are not pruned.
- each individual tree has high variance, but low bias.
- Averaging these B trees reduces the variance.
- How to perform bagging on classification problems?
 - For a given test observation, we can record the class predicted by each of the B trees, and take a *majority vote*: the overall prediction is the most commonly occurring class among the B predictions.

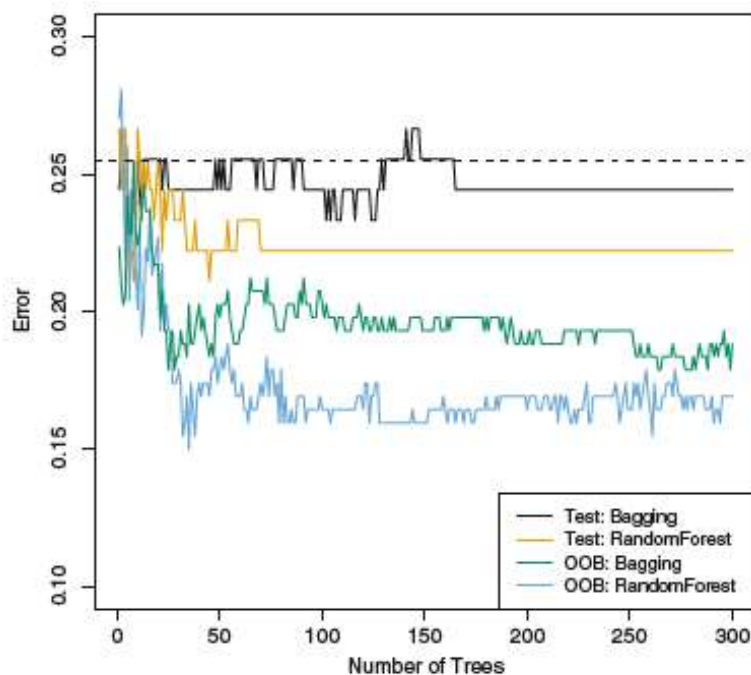


FIGURE 8.8. Bagging and random forest results for the **Heart** data. The test error (black and orange) is shown as a function of B , the number of bootstrapped training sets used. Random forests were applied with $m = \sqrt{p}$. The dashed line indicates the test error resulting from a single classification tree. The green and blue traces show the OOB error, which in this case is considerably lower.

- when bagging → it is no longer possible to represent the resulting statistical learning procedure using a single tree
- and it is no longer clear which variables are most important to the procedure.
- Thus, bagging improves prediction accuracy at the expense of interpretability
- What can be done?
 - summary of the importance of each predictor using the RSS (for bagging regression trees) or the Gini index (for bagging classification trees)

- In the case of bagging regression trees → record the total amount that the RSS is decreased due to splits over a given predictor, averaged over all B trees.
- A large value indicates an important predictor.
- in the context of bagging classification trees → add up the total amount that the Gini index is decreased by splits over a given predictor, averaged over all B trees.
- graphical representation of the *variable importances*

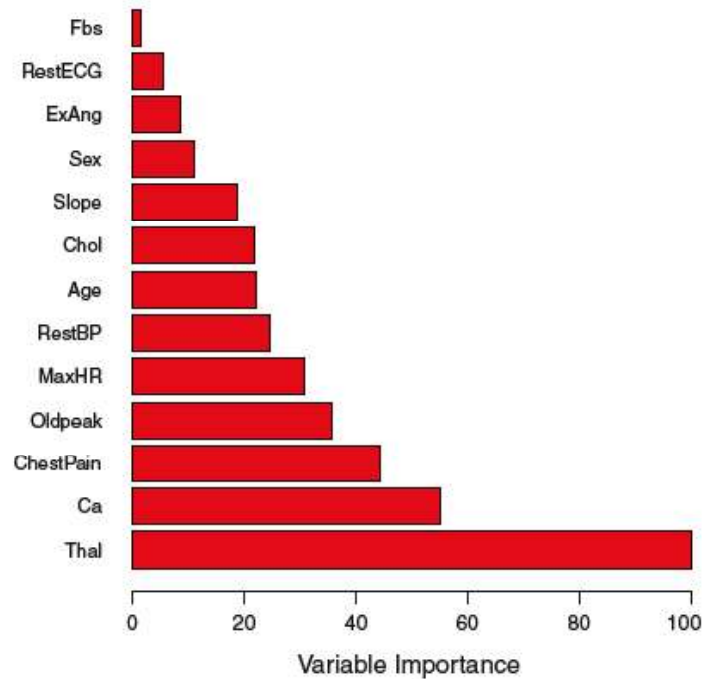


FIGURE 8.9. A variable importance plot for the **Heart** data. Variable importance is computed using the mean decrease in Gini index, and expressed relative to the maximum.

8.2.2 Random Forests

- provide an improvement over bagged trees by way of a random small tweak that *decorrelates* the trees
- build a number of decision trees on bootstrapped training samples.
- each time a split in a tree is considered, *a random sample of m predictors* is chosen as split candidates from the full set of p predictors.
- The split is allowed to use only one of those m predictors.
- A fresh sample of m predictors is taken at each split
 - typically $m \approx \sqrt{p}$
- If $m = p \rightarrow$ random forest = bagging

- small value of m will be helpful when we have a large number of correlated predictors

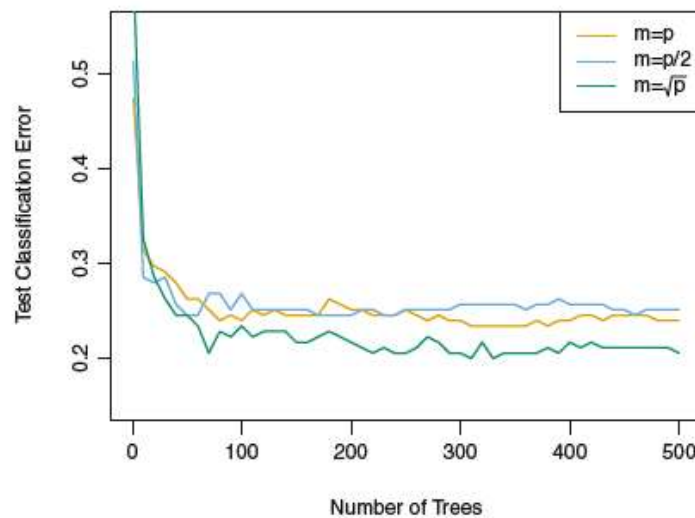


FIGURE 8.10. Results from random forests for the 15-class gene expression data set with $p = 500$ predictors. The test error is displayed as a function of the number of trees. Each colored line corresponds to a different value of m , the number of predictors available for splitting at each interior tree node. Random forests ($m < p$) lead to a slight improvement over bagging ($m = p$). A single classification tree has an error rate of 45.7%.

8.3 Lab: Decision Trees

References:

→ from textbook: cap. 2 – Statistical learning

K-Nearest Neighbors

- In theory we would always like to predict qualitative responses using the Bayes classifier.
- But for real data, we do not know the conditional distribution of Y given X , and so **computing the Bayes classifier is impossible**.
- **KNN** → estimate the conditional distribution of Y given X , and then classify a given observation to the class with highest **estimated** probability

$$\Pr(Y = j|X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j)$$

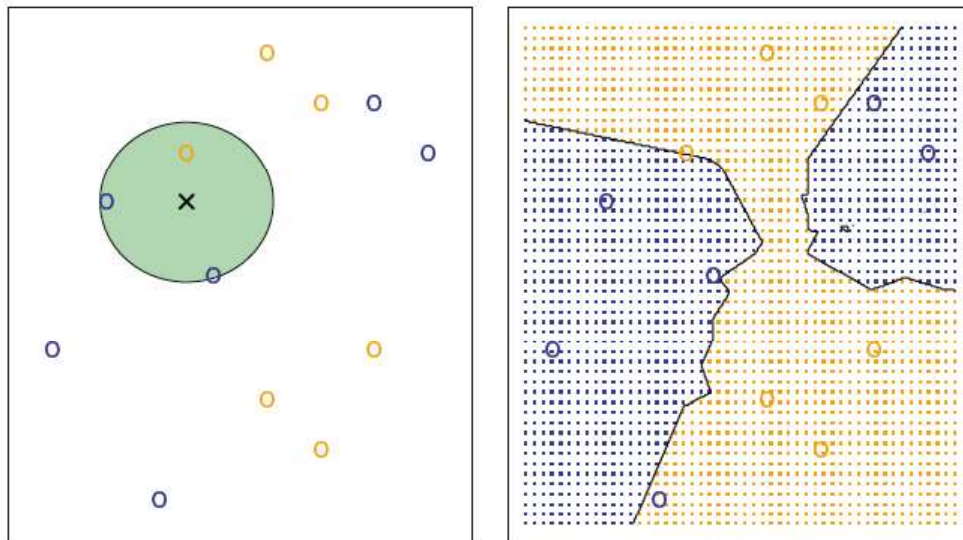


FIGURE 2.14. The KNN approach, using $K = 3$, is illustrated in a simple situation with six blue observations and six orange observations. Left: a test observation at which a predicted class label is desired is shown as a black cross. The three closest points to the test observation are identified, and it is predicted that the test observation belongs to the most commonly-occurring class, in this case blue. Right: The KNN decision boundary for this example is shown in black. The blue grid indicates the region in which a test observation will be assigned to the blue class, and the orange grid indicates the region in which it will be assigned to the orange class.

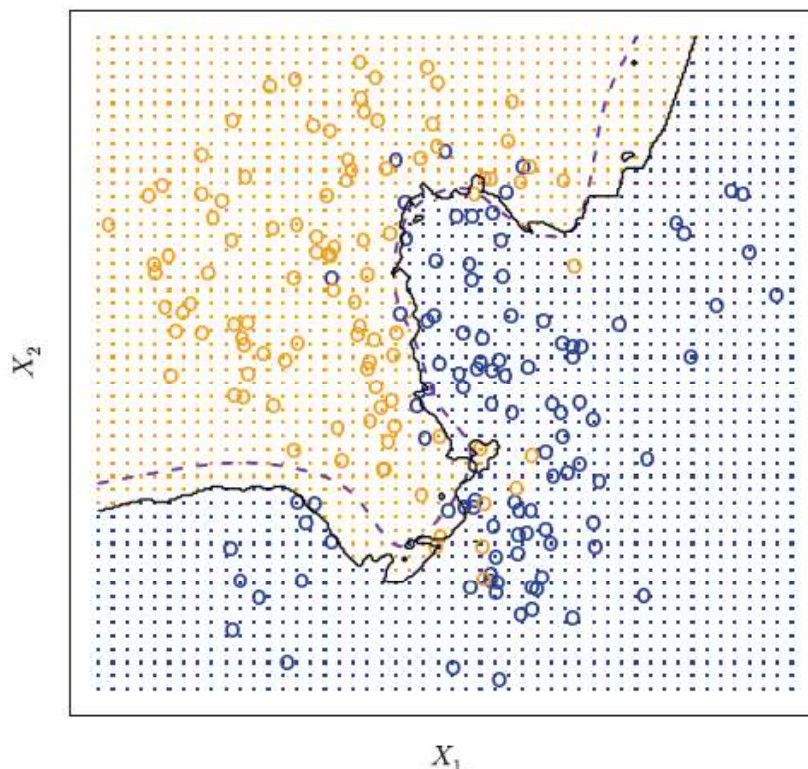


FIGURE 2.15. The black curve indicates the KNN decision boundary on the data from Figure 2.13, using $K = 10$. The Bayes decision boundary is shown as a purple dashed line. The KNN and Bayes decision boundaries are very similar.

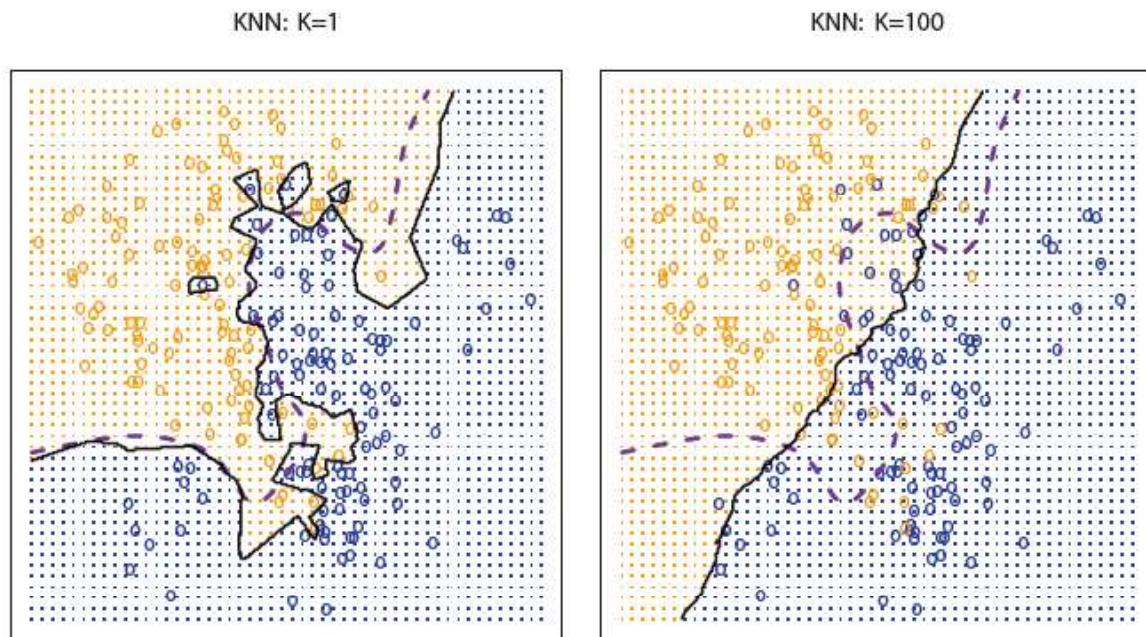


FIGURE 2.16. A comparison of the KNN decision boundaries (solid black curves) obtained using $K = 1$ and $K = 100$ on the data from Figure 2.13. With $K = 1$, the decision boundary is overly flexible, while with $K = 100$ it is not sufficiently flexible. The Bayes decision boundary is shown as a purple dashed line.

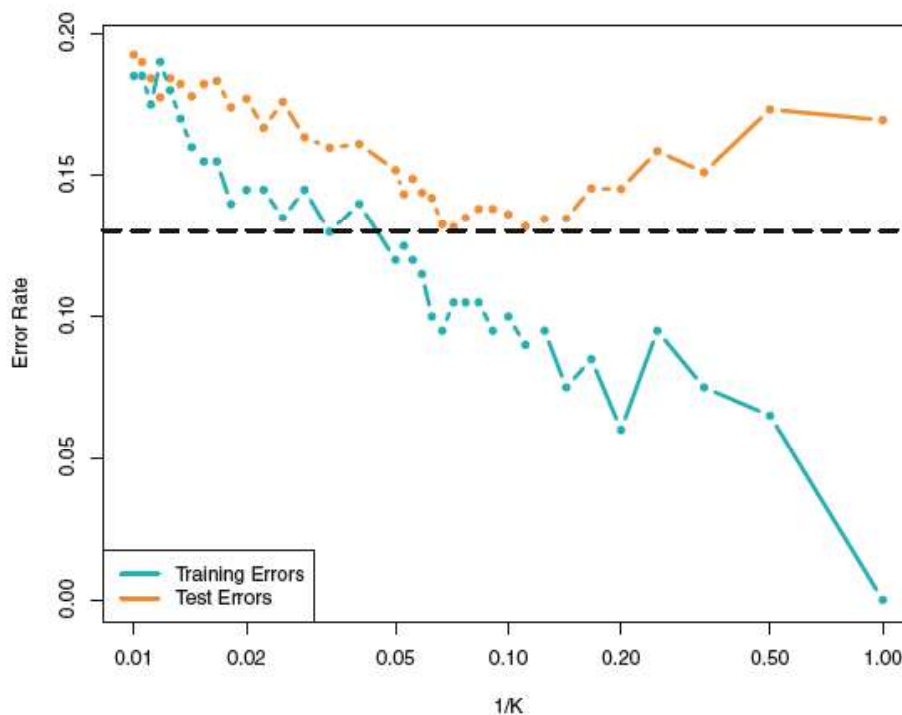


FIGURE 2.17. The KNN training error rate (blue, 200 observations) and test error rate (orange, 5,000 observations) on the data from Figure 2.13, as the level of flexibility (assessed using $1/K$) increases, or equivalently as the number of neighbors K decreases. The black dashed line indicates the Bayes error rate. The jumpiness of the curves is due to the small size of the training data set.

4.6 Lab: ~~Logistic Regression, LDA, QDA, and~~ KNN